23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : _____

Team Members :

Roll No	Student Name
AM.SC.U4CSE25245	Saharsh Baiju

Project Title : **O(1)** — an ICPC preparation tracker for competitive-programming teams and their mentors

3.1. Detailed description of the project

Competitive-programming teams preparing for the ICPC train across several websites at once: they solve problems on Codeforces, follow topic ladders a coach hands out, run past regional problem sets as timed practice, and keep a spreadsheet of who has solved what. Nothing joins these together, so a mentor cannot see at a glance which student is behind on which topic, and a student cannot see what to work on next. O(1) is a web application that gives a college's ICPC teams one place to practise, be assigned work by a mentor, and measure progress against each other.

Users

The system has two kinds of user, and a person holds exactly one role:

- **Student**: a contestant. Links a judge account, works through assigned practice sheets, belongs to at most one team, and views their own analytics, the team's combined coverage, and the leaderboard.
- **Mentor**: a faculty member or senior who coaches. Everything a student can do, plus authoring practice sheets, assigning them to students, coaching teams, and adding learning resources.

Entities and what the system records about them

- **University.** Every user and every team belongs to one university. Leaderboards can be filtered by university.
- **User.** Name, unique e-mail, password hash, role, year of study, university, join date, short bio and avatar.

- **Judge account.** A user's handle on an online judge (initially Codeforces; the design allows AtCoder later). A user has at most one account per platform, a handle is unique within a platform, and the account carries the judge-side rating and the time the handle was verified. It cannot exist without its owner.
- **Problem.** A problem hosted on a judge (identified there by a platform-specific code such as `1234C`) or a manually added link. It has a title, a numeric rating, a URL, and a flag for whether an editorial exists. A problem is tagged with any number of **topics** (Graphs, Dynamic Programming, Flows, …).
- **Contest.** A rated round, a past-ICPC set mirrored in the Codeforces Gym, or a practice set. It has a name, kind, start time and duration, and includes an ordered set of problems labelled A, B, C, ….
- **Submission.** One attempt by a user at a problem, pulled from the judge by a periodic sync job: verdict, language, timestamp, and the judge's own submission id. A submission made during a contest is linked to that contest; a practice submission is not.
- **Participation.** A user's result in a rated contest: old rating, new rating and rank. This is the source of the rating-history chart.
- **Practice sheet.** A mentor-authored, ordered list of problems with a level, a description and an optional deadline. A sheet may instead point at a past-ICPC contest, in which case its problems are that contest's problems. Sheets are assigned to students; a student's progress is the set of sheet problems they have solved (detected automatically from submissions) or ticked off by hand (for problems the sync cannot see).
- **Team.** An ICPC squad of at most three students from one university, coached by a mentor, with an invite code that lets a student join.
- **Learning resource.** An article, video, notes, roadmap, tutorial or book for one topic and level, added by a mentor. Users can bookmark resources.
- **Calendar event.** A practice session, team meeting or announcement, either personal or belonging to a team. Users may set a reminder on an event. Contest dates and sheet deadlines also appear on the calendar but are not stored as events, because they are already recorded on the contest and sheet.

Business rules

1. A user has exactly one role (student or mentor) and belongs to exactly one university.
2. A user has at most one judge account per platform; a handle is unique on its platform.
3. A problem is uniquely identified on its platform by its external code; the same problem can appear in several contests (for example a Div. 1 and Div. 2 mirror).
4. Within a contest every problem has a distinct label (A, B, C …).
5. Every submission belongs to exactly one user and one problem. Contest submissions reference their contest; practice submissions do not.
6. A user has at most one participation record per contest.
7. A sheet has exactly one author, who must be a mentor. The problems in a sheet are ordered, and each position is used once.
8. A student can be assigned a sheet only once, and can mark complete only problems that belong to a sheet they are assigned.
9. A team has exactly one coach (a mentor), belongs to one university, and has at most three members. A student is a member of at most one team.

10. A resource covers exactly one topic and is added by one user.
11. A reminder belongs to one user and one event; a user has at most one reminder per event.
12. Counts and statuses that can be computed (problems solved, streak, sheet progress, contest status, difficulty band, leaderboard rank) are never stored; they are derived by queries so that they cannot become stale.

3.2. ER Diagram

The diagrams use Chen notation. Rectangles are entities (a double border marks the weak entity *JudgeAccount*); diamonds are relationships (double border for the identifying relationship *links*); ellipses are attributes, with primary-key attributes underlined and derived attributes dashed. Cardinality is written on the edges as 1, N or M, and a double line marks total participation. To keep the figure legible, only the principal attributes of each entity are drawn; the complete attribute list is in the tables that follow and in the relational schema in §3.3.

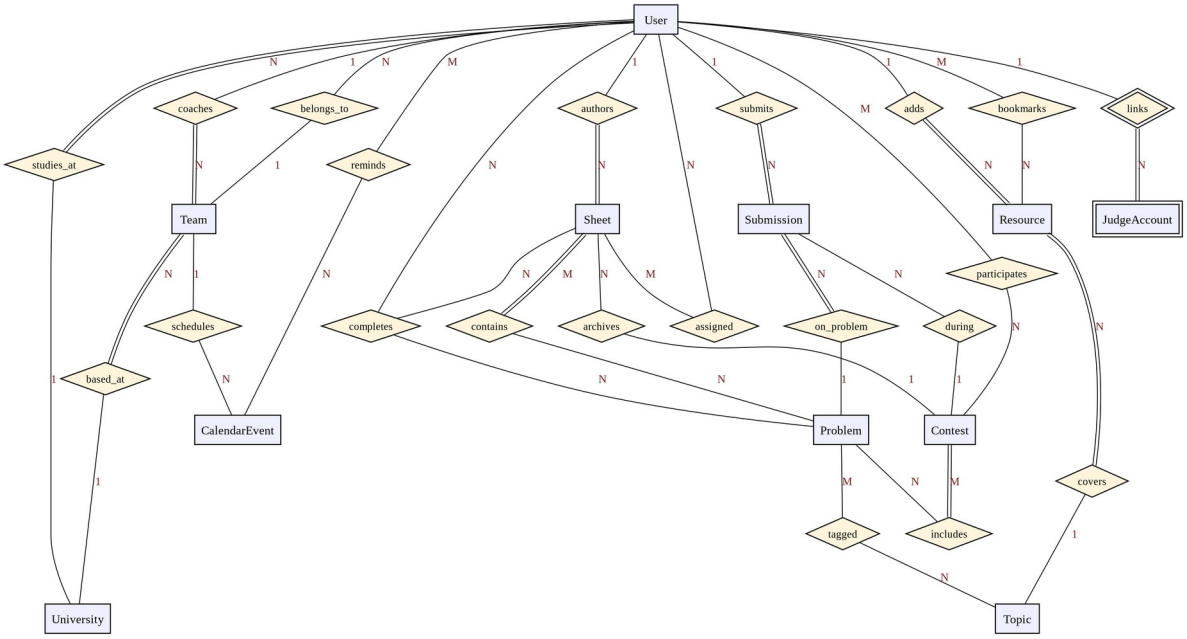


Figure 1 — Entities and relationships of O(1) with cardinality and participation (attributes omitted for legibility). Source: `docs/er-overview.dot`.

Figure 2 on the next page is the complete Chen diagram with attributes, rotated to fit the page; a zoomable copy is in `docs/er-diagram.svg`.

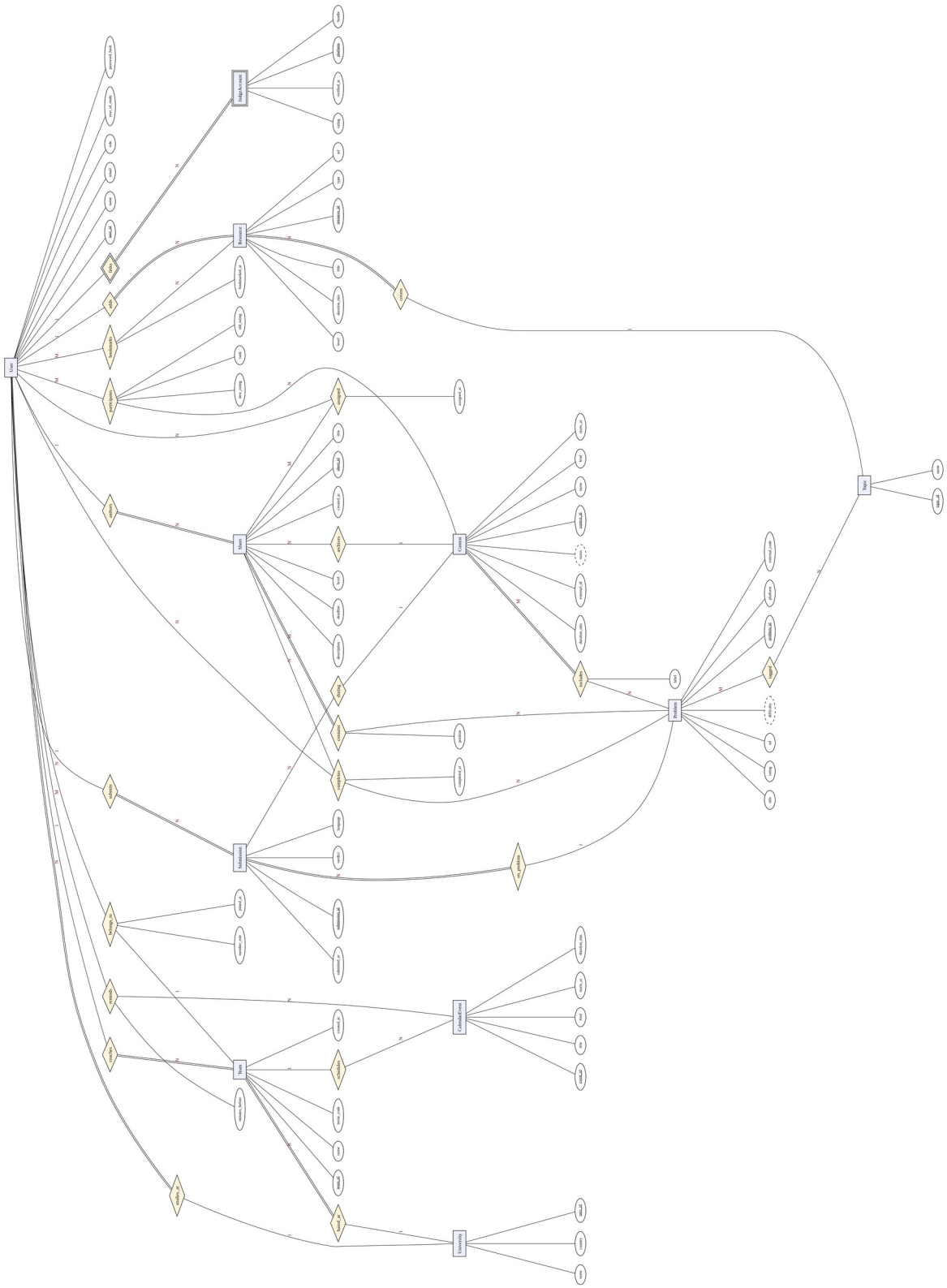


Figure 2 — Full ER diagram (Chen notation), rotated 90°. Source: `docs/er-diagram.dot`.

Entity summary

Entity	Key	Attributes
University	univ_id	name, country
User	user_id	name, email, password_hash, role, year_of_study, joined_at, bio,

		avatar_url
JudgeAccount *(weak)*	platform (partial key) + owner	handle, rating, verified_at
Topic	topic_id	name
Problem	problem_id	platform, external_code, title, rating, url, has_editorial, *difficulty (derived from rating)*
Contest	contest_id	platform, external_id, name, kind, starts_at, duration_min, *status (derived from clock)*
Submission	submission_id	verdict, language, submitted_at, platform, external_submission_id
Sheet	sheet_id	title, description, level, deadline, created_at
Team	team_id	name, invite_code, created_at
Resource	resource_id	title, type, url, level, duration_min
CalendarEvent	event_id	title, kind, starts_at, duration_min

Relationship summary

Relationship	**Entities**	**Cardinality**	**Participation**	**Attributes / rule**
studies_at	User – University	N : 1	User total, University partial	—
based_at	Team – University	N : 1	Team total	—
coaches	User (mentor) – Team	1 : N	Team total	coach must have role = mentor
belongs_to	User – Team	N : 1	both partial	member_role, joined_at; ≤ 3 members per team
links *(identifying)*	User – JudgeAccount	1 : N	JudgeAccount total	—
tagged	Problem – Topic	M : N	both partial	—
includes	Contest – Problem	M : N	Contest total	label, unique within contest
submits	User – Submission	1 : N	Submission total	—
on_problem	Submission – Problem	N : 1	Submission total	—
during	Submission – Contest	N : 1	both partial	absent for practice submissions
participates	User – Contest	M : N	both partial	old_rating, new_rating, rank
authors	User (mentor) – Sheet	1 : N	Sheet total	—

contains	Sheet – Problem	M : N	Sheet total	position, unique within sheet
assigned	Sheet – User	M : N	both partial	assigned_at
completes *(ternary)*	User – Sheet – Problem	N : N : N	all partial	completed_at; only for assigned sheet problems
archives	Sheet – Contest	N : 1	both partial	set only for past-ICPC sheets
covers	Resource – Topic	N : 1	Resource total	—
adds	User – Resource	1 : N	Resource total	—
bookmarks	User – Resource	M : N	both partial	bookmarked_at
schedules	Team – CalendarEvent	1 : N	both partial	absent for personal events
reminds	User – CalendarEvent	M : N	both partial	minutes_before

3.3. Relational Schema

The ER model is mapped to relations with the standard rules:

- **Strong entity** → one relation with the entity's key as primary key.
- **Weak entity** (JudgeAccount) → a relation whose primary key is the owner's key plus the partial key.
- **1 : N relationship** → the key of the "1" side becomes a foreign key on the "N" side (studies_at, based_at, coaches, on_problem, submits, during, authors, archives, covers, adds, schedules). Where participation of the N side is partial the foreign key is nullable (during, archives, schedules).
- **N : 1 relationship with attributes** whose N side has at most one instance (belongs_to) → a separate relation keyed by the N side, because a user's membership row is optional and carries its own attributes.
- **M : N relationship** → a junction relation whose primary key is the union of the two keys, plus the relationship's attributes (tagged, includes, participates, contains, assigned, bookmarks, reminds).
- **Ternary relationship** (completes) → a relation keyed by all three participating keys.
- **Derived attributes** (difficulty, status) are not stored; they are views (§3.5).

Primary keys are underlined; foreign keys are marked with → and the relation they reference.

People and lookups

University(univ_id, name, country)

Topic(topic_id, name)

User(user_id, name, email, password_hash, role, year_of_study, univ_id → University, joined_at, bio, avatar_url)

JudgeAccount(user_id → User, platform, handle, rating, verified_at)

Problems and contests

Problem(problem_id, platform, external_code, title, rating, url, has_editorial)

ProblemTopic(problem_id → Problem, topic_id → Topic)

Contest(contest_id, platform, external_id, name, kind, starts_at, duration_min)

ContestProblem(contest_id → Contest, problem_id → Problem, label)

Submission(submission_id, user_id → User, problem_id → Problem, contest_id → Contest *[null]*, verdict, language, submitted_at, platform, external_submission_id)

Participation(user_id → User, contest_id → Contest, old_rating, new_rating, rank)

Practice sheets

Sheet(sheet_id, title, description, author_id → User, level, deadline, created_at, archive_contest_id → Contest *[null]*)

SheetProblem(sheet_id → Sheet, problem_id → Problem, position)

SheetAssignment(sheet_id → Sheet, user_id → User, assigned_at)

SheetCompletion(sheet_id, user_id, problem_id, completed_at) *(sheet_id, problem_id) → SheetProblem; (sheet_id, user_id) → SheetAssignment*

Teams

Team(team_id, name, univ_id → University, coach_id → User, invite_code, created_at)

TeamMember(user_id → User, team_id → Team, member_role, joined_at)

Resources and calendar

Resource(resource_id, title, type, url, topic_id → Topic, level, duration_min, added_by → User)

Bookmark(user_id → User, resource_id → Resource, bookmarked_at)

CalendarEvent(event_id, title, kind, starts_at, duration_min, team_id → Team *[null]*, created_by → User)

Reminder(user_id → User, event_id → CalendarEvent, minutes_before)

Additional constraints

Relation	Constraint	Enforces rule
User	UNIQUE(email); role ∈ {student, mentor}	1
JudgeAccount	UNIQUE(platform, handle)	2
Problem	UNIQUE(platform, external_code); 800 ≤ rating ≤ 3500	3
Contest	UNIQUE(platform, external_id); kind ∈ {rated, gym, practice}	—
ContestProblem	UNIQUE(contest_id, label)	4
Submission	UNIQUE(platform, external_submission_id)	5 (idempotent sync)
Sheet	author_id must reference a user with role = mentor (trigger)	7
SheetProblem	UNIQUE(sheet_id, position)	7
SheetCompletion	composite FKs to SheetProblem and SheetAssignment	8
Team	UNIQUE(invite_code); UNIQUE(univ_id, name); coach role = mentor (trigger)	9
TeamMember	PRIMARY KEY(user_id); trigger rejects a fourth member	9
Resource	UNIQUE(url); type ∈ {Article, Video, Notes, Roadmap, Tutorial, Book}	10

3.4. Normalization and Functional Dependency Analysis

3.4.1 Functional dependencies

The FDs below are the ones that hold because of the business rules in §3.1. Trivial dependencies are omitted. Attributes are abbreviated where the meaning is obvious.

Relation	Functional dependencies	Meaning
University	`univ_id → name, country` `name → univ_id`	university names are unique
Topic	`topic_id → name` `name → topic_id`	topic names are unique
User	`user_id → name, email, password_hash, role, year, univ_id, joined_at, bio, avatar_url` `email → user_id`	e-mail is the login and is unique (rule 1)
JudgeAccount	`user_id, platform → handle, rating, verified_at` `platform, handle → user_id`	one account per platform per user; handle unique per platform (rule 2)
Problem	`problem_id → platform, external_code, title, rating, url, has_editorial` `platform, external_code → problem_id`	a judge identifies a problem by its code (rule 3)
ProblemTopic	`none beyond the key`	all-key relation
Contest	`contest_id → platform, external_id, name, kind, starts_at, duration_min` `platform, external_id → contest_id`	judge's contest id is unique per platform
ContestProblem	`contest_id, problem_id → label` `contest_id, label → problem_id`	labels are unique within a contest (rule 4)
Submission	`submission_id → user_id, problem_id, contest_id, verdict, language, submitted_at, platform, ext_sub_id` `platform, ext_sub_id → submission_id`	judge's submission id is unique per platform (rule 5)
Participation	`user_id, contest_id → old_rating, new_rating, rank`	one result per user per contest (rule 6)
Sheet	`sheet_id → title, description, author_id, level, deadline, created_at, archive_contest_id`	rule 7
SheetProblem	`sheet_id, problem_id → position` `sheet_id, position → problem_id`	each position used once (rule 7)
SheetAssignment	`sheet_id, user_id → assigned_at`	rule 8

SheetCompletion	`sheet_id, user_id, problem_id → completed_at`	rule 8

Team	`team_id → name, univ_id, coach_id, invite_code, created_at`
	`invite_code → team_id`
	`univ_id, name → team_id`
		invite code unique; team name unique within a university (rule 9)

TeamMember	`user_id → team_id, member_role, joined_at`	a student is in at most one team (rule 9)
Resource	`resource_id → title, type, url, topic_id, level, duration_min, added_by` `url → resource_id`	rule 10
Bookmark	`user_id, resource_id → bookmarked_at`	—
CalendarEvent	`event_id → title, kind, starts_at, duration_min, team_id, created_by`	—
Reminder	`user_id, event_id → minutes_before`	one reminder per user per event (rule 11)

Why the FDs matter. Each non-trivial FD is either a primary-key dependency or a uniqueness rule from §3.1. The second kind (email → user_id, (platform, handle) → user_id, (platform, external_code) → problem_id, (contest_id, label) → problem_id, (sheet_id, position) → problem_id, invite_code → team_id, url → resource_id) tells us each of those relations has more than one candidate key. That is exactly the situation in which 3NF and BCNF can differ, so those relations are checked most carefully in §3.4.4. Dependencies that are deliberately *absent* also matter: rating does not determine a stored difficulty, and a problem does not determine a contest, because a problem can be shared by two contests (rule 3). Both would otherwise have created redundancy.

3.4.2 Candidate keys by attribute closure

For each relation the candidate keys are the minimal attribute sets whose closure under the FDs is the whole relation. The closures below are computed for the relations where the answer is not immediate.

Problem

R = {problem_id, platform, external_code, title, rating, url, has_editorial}.

- $\{problem_id\}^+ = \{problem_id\} \cup \{platform, external_code, title, rating, url, has_editorial\} = R$ → candidate key.
- $\{platform, external_code\}^+ = \{platform, external_code, problem_id\}$ → then apply problem_id → … = R → candidate key.
- $\{platform\}^+ = \{platform\}$; $\{external_code\}^+ = \{external_code\}$; $\{rating\}^+ = \{rating\}$ → not keys, so both keys above are minimal.

Candidate keys: {problem_id}, {platform, external_code}. Primary key: problem_id (shorter, stable if a judge renames a code). Prime attributes: problem_id, platform, external_code.

JudgeAccount

R = {user_id, platform, handle, rating, verified_at}.

- $\{user_id, platform\}^+ = \{user_id, platform, handle, rating, verified_at\} = R \rightarrow$ candidate key.
- $\{platform, handle\}^+ = \{platform, handle, user_id\} \rightarrow \{user_id, platform\} \rightarrow R \rightarrow$ candidate key.
- $\{user_id\}^+ = \{user_id\}$; $\{handle\}^+ = \{handle\} \rightarrow$ neither key is reducible.

Candidate keys: {user_id, platform}, {platform, handle}. Primary key: (user_id, platform), because it is the identifying relationship of the weak entity. All attributes except rating and verified_at are prime.

Submission

R = {submission_id, user_id, problem_id, contest_id, verdict, language, submitted_at, platform, ext_sub_id}.

- $\{submission_id\}^+ = R \rightarrow$ candidate key.
- $\{platform, ext_sub_id\}^+ = \{platform, ext_sub_id, submission_id\} \rightarrow R \rightarrow$ candidate key.
- $\{user_id, problem_id, submitted_at\}^+ = \{user_id, problem_id, submitted_at\}$. A user can submit the same problem twice in one second, so this is *not* a key, which is why the surrogate id is needed.

Candidate keys: {submission_id}, {platform, ext_sub_id}. Primary key: submission_id.

TeamMember

R = {user_id, team_id, member_role, joined_at}.

- $\{user_id\}^+ = \{user_id, team_id, member_role, joined_at\} = R \rightarrow$ candidate key.
- $\{team_id, user_id\}^+ = R$, but it contains the key {user_id}, so it is a superkey, not a candidate key.
- $\{team_id\}^+ = \{team_id\}$; $\{team_id, member_role\}^+ = \{team_id, member_role\} \rightarrow$ not keys (two members of a team may share a role label).

Candidate key: {user_id}. This is why the relation is keyed by user_id alone rather than by (team_id, user_id), which the M : N mapping rule would have produced for a relationship without the "one team per student" rule.

ContestProblem and SheetProblem

Both have the same shape, R = {X, Y, Z} with X,Y → Z and X,Z → Y.

- $\{contest_id, problem_id\}^+ = R$; $\{contest_id, label\}^+ = \{contest_id, label, problem_id\} = R$.
- $\{contest_id\}^+ = \{contest_id\}$; $\{problem_id\}^+ = \{problem_id\}$; $\{label\}^+ = \{label\}$.

Candidate keys: {contest_id, problem_id} and {contest_id, label}; likewise {sheet_id, problem_id} and {sheet_id, position}. Every attribute of these relations is prime.

User, Team, Resource, University, Topic, Contest

Each has a surrogate id whose closure is the whole relation, plus one natural key from a uniqueness rule (email; invite_code and (univ_id, name); url; name; name; (platform, external_id)). The closure of each natural key reaches the surrogate id in one step and then the rest of the relation, so each is a second candidate key. The surrogate is chosen as primary key in every case.

All-key and single-key relations

ProblemTopic has only trivial FDs, so its whole heading is the only candidate key. Participation, SheetAssignment, SheetCompletion, Bookmark, Reminder, Sheet and CalendarEvent each have exactly one non-trivial determinant, which is therefore the only candidate key.

3.4.3 Normalization from 1NF to BCNF

The starting point is the way the data was actually kept before this design: the front-end prototype stores everything in a handful of denormalised records, and a coach tracks sheet progress in a spreadsheet with one row per (sheet, student, problem). Taken together they amount to a single universal relation for sheet tracking:

`PROGRESS(sheet_id, sheet_title, deadline, mentor_id, mentor_name, mentor_email, student_id, student_name, student_email, univ_id, univ_name, team_id, team_name, coach_id, problem_id, problem_title, platform, external_code, rating, difficulty, topics, completed_at)`

A sample of the data as it would be entered (some columns omitted for width):

sheet_id	sheet_title	mentor_name	student_id	student_name	team_name	problem_id	rating	difficulty	topics
s-01	Graph Theory Foundations	Aarav Mehta	u-001	Saharsh Baiju	Segfault Sentinels	p-118	2100	Hard	{Trees, Data Structure }
s-01	Graph Theory Foundations	Aarav Mehta	u-001	Saharsh Baiju	Segfault Sentinels	p-042	1700	Medium	{Graphs}
s-01	Graph Theory Foundations	Aarav Mehta	u-005	Ananya Ghosh	Segfault Sentinels	p-118	2100	Hard	{Trees, Data Structure }

The functional dependencies that hold on PROGRESS are:

- `F1: sheet_id → sheet_title, deadline, mentor_id`
- `F2: mentor_id → mentor_name, mentor_email`
- `F3: student_id → student_name, student_email, univ_id, team_id`
- `F4: univ_id → univ_name`
- `F5: team_id → team_name, coach_id`
- `F6: problem_id → problem_title, platform, external_code, rating`
- `F7: platform, external_code → problem_id`
- `F8: rating → difficulty`
- `F9: sheet_id, student_id, problem_id → completed_at`

Closure: {sheet_id, student_id, problem_id}$^+$ reaches every attribute through F1–F9, and no proper subset does (sheet_id alone never reaches student attributes, and so on). So the candidate key is **(sheet_id, student_id, problem_id)** and the prime attributes are exactly those three, plus platform and external_code, which can substitute for problem_id via F7.

Unnormalised → 1NF

Violation. `topics` is multivalued: one cell holds a set. 1NF requires every attribute value to be atomic.

Fix. Move the multivalued attribute into its own relation keyed by the problem and the topic.

Repeating the PROGRESS row once per topic would also give 1NF but would multiply every other column by the number of topics, so the split is preferred.

PROGRESS$_1$(sheet_id, student_id, problem_id, sheet_title, deadline, mentor_id, mentor_name, mentor_email, student_name, student_email, univ_id, univ_name, team_id, team_name, coach_id, problem_title, platform, external_code, rating, difficulty, completed_at)

PROBLEM_TOPIC(problem_id, topic)

1NF → 2NF

Violation. 2NF forbids a non-prime attribute depending on only part of a candidate key. F1, F3 and F6 are exactly such partial dependencies: sheet_title depends on sheet_id alone, student_name on student_id alone, problem_title on problem_id alone. The consequence is visible in the sample: "Graph Theory Foundations" and "Aarav Mehta" are repeated on every row of the sheet, and changing a sheet's deadline means updating one row per assigned student per problem (update anomaly). A sheet with no student assigned yet cannot be recorded at all (insertion anomaly).

Fix. Project each partial dependency into its own relation, keeping only the full-key dependency F9 in the original.

SHEET$_2$(sheet_id, sheet_title, deadline, mentor_id, mentor_name, mentor_email)

STUDENT$_2$(student_id, student_name, student_email, univ_id, univ_name, team_id, team_name, coach_id)

PROBLEM$_2$(problem_id, problem_title, platform, external_code, rating, difficulty)

COMPLETION(sheet_id, student_id, problem_id, completed_at)

PROBLEM_TOPIC(problem_id, topic)

COMPLETION is now all-key plus one attribute that depends on the whole key, so it is in 2NF. Note that COMPLETION still records *assignment* only implicitly: the row with `completed_at = null` is what says Saharsh is assigned problem p-042. That is resolved in the 3NF step below.

2NF → 3NF

Violation. 3NF forbids a non-prime attribute depending transitively on the key through another non-prime attribute. Three remain:

- In SHEET$_2$: sheet_id → mentor_id → mentor_name, mentor_email (F2). A mentor's name is stored once per sheet they wrote.
- In STUDENT$_2$: student_id → univ_id → univ_name (F4) and student_id → team_id → team_name, coach_id (F5). Renaming a team means editing every member's row, and a team with no members yet cannot exist.
- In PROBLEM$_2$: problem_id → rating → difficulty (F8). difficulty is a pure function of rating; storing it lets the two disagree.

Fix. Project each transitive dependency out, leaving the foreign key behind.

SHEET$_3$(sheet_id, sheet_title, deadline, mentor_id → MENTOR)

MENTOR(mentor_id, mentor_name, mentor_email)

STUDENT$_3$(student_id, student_name, student_email, univ_id → UNIVERSITY, team_id → TEAM)

```
UNIVERSITY(univ_id, univ_name)
TEAM(team_id, team_name, coach_id → MENTOR)
PROBLEM3(problem_id, problem_title, platform, external_code, rating)
DIFFICULTY_BAND(rating, difficulty)   — or, as chosen in the final design, a view computing the band from rating
COMPLETION(sheet_id, student_id, problem_id, completed_at)
PROBLEM_TOPIC(problem_id, topic)
```

MENTOR and $STUDENT_3$ have the same attributes apart from role, and a mentor can also be a contestant's coach and a team's coach, so the final design merges them into one **User** relation with a `role` attribute. The membership attributes are moved out of STUDENT into **TeamMember**, keyed by user_id, so that a user without a team does not need a null team_id and the membership can carry its own attributes (member_role, joined_at). Finally the null-valued rows of COMPLETION are replaced by an explicit **SheetAssignment** relation and an explicit **SheetProblem** relation, so that "assigned" and "in the sheet" are facts in their own right rather than implied by a null.

3NF → BCNF

BCNF requires that for *every* non-trivial FD X → Y, X is a superkey, including FDs whose right-hand side is a prime attribute, which 3NF lets through. The relations with more than one candidate key are the ones to check.

A genuine BCNF violation in the first draft of the assignment roster. The mentor roster page was first modelled directly from the screen, which lists, per student, the sheet they are on and the mentor who set it:

```
ROSTER(student_id, sheet_id, mentor_id)
```

with the draft rule that a mentor keeps one sheet open per student at a time. The FDs are:

- `G1: sheet_id → mentor_id  (a sheet has one author)`
- `G2: student_id, mentor_id → sheet_id  (one open sheet per mentor per student)`

Closures: $\{student_id, sheet_id\}^+ = R$ via G1, and $\{student_id, mentor_id\}^+ = R$ via G2 then G1. Both are candidate keys, and they overlap. Every attribute is prime, so the relation is trivially in 3NF. But G1 has determinant sheet_id, which is not a superkey ($\{sheet_id\}^+ = \{sheet_id, mentor_id\} \neq R$). ROSTER is therefore **not in BCNF**, and it shows the classic symptom: the author of sheet s-01 is repeated once for every student assigned to it, and if the last student is unassigned the sheet's author is lost.

Decomposition on the offending FD, $R \rightarrow (X \cup Y)$ and $(R - Y)$:

```
SHEET_AUTHOR(sheet_id, mentor_id)   → absorbed into Sheet.author_id
SHEET_ASSIGNMENT(student_id, sheet_id)
```

Both projections are in BCNF (each has a single candidate key that is the determinant of its only FD), the join is lossless because the common attribute sheet_id is a key of SHEET_AUTHOR, and the drafting rule G2 is no longer needed as a dependency: it can be imposed later by a partial unique index if mentors want it, or dropped, which is what the final design does.

Relations with two candidate keys that are already in BCNF. The remaining multi-key relations all have the property that every determinant is itself a candidate key:

Relation	Determinants of its non-trivial FDs	Each a superkey?
User	`{user_id}, {email}`	yes → BCNF
JudgeAccount	`{user_id, platform}, {platform, handle}`	yes → BCNF
Problem	`{problem_id}, {platform, external_code}`	yes → BCNF
Contest	`{contest_id}, {platform, external_id}`	yes → BCNF
ContestProblem	`{contest_id, problem_id}, {contest_id, label}`	yes → BCNF
Submission	`{submission_id}, {platform, ext_sub_id}`	yes → BCNF
SheetProblem	`{sheet_id, problem_id}, {sheet_id, position}`	yes → BCNF
Team	`{team_id}, {invite_code}, {univ_id, name}`	yes → BCNF
Resource	`{resource_id}, {url}`	yes → BCNF
University, Topic	`{id}, {name}`	yes → BCNF

Two places where a BCNF violation was avoided by design rather than by decomposition.

- Submission does *not* store the contest label of the problem. Had it done so, the FD (contest_id, problem_id) → label would hold inside Submission with a determinant that is not a superkey. The label lives only in ContestProblem.
- TeamMember does *not* store coach_id or univ_id. Had it done so, team_id → coach_id would hold with team_id not a superkey. Those attributes live only in Team.

3.4.4 Final normal-form check

Relation	Candidate key(s)	Determinants all superkeys?	Form
University	univ_id; name	yes	BCNF
Topic	topic_id; name	yes	BCNF
User	user_id; email	yes	BCNF
JudgeAccount	(user_id, platform); (platform, handle)	yes	BCNF
Problem	problem_id; (platform, external_code)	yes; difficulty not stored	BCNF
ProblemTopic	(problem_id, topic_id)	no non-trivial FDs	BCNF
Contest	contest_id; (platform, external_id)	yes; status not stored	BCNF
ContestProblem	(contest_id, problem_id); (contest_id, label)	yes	BCNF
Submission	submission_id; (platform, ext_sub_id)	yes; label not stored	BCNF
Participation	(user_id, contest_id)	yes	BCNF
Sheet	sheet_id	yes; author details not stored	BCNF
SheetProblem	(sheet_id, problem_id); (sheet_id, position)	yes	BCNF
SheetAssignment	(sheet_id, user_id)	yes; mentor not stored	BCNF
SheetCompletion	(sheet_id, user_id, problem_id)	yes	BCNF

Team	team_id; invite_code; (univ_id, name)	yes	BCNF
TeamMember	user_id	yes; coach/university not stored	BCNF
Resource	resource_id; url	yes	BCNF
Bookmark	(user_id, resource_id)	yes	BCNF
CalendarEvent	event_id	yes	BCNF
Reminder	(user_id, event_id)	yes	BCNF

All twenty relations are in BCNF. Every decomposition performed above was on a functional dependency whose left-hand side is a key of one of the resulting relations, so each is lossless, and since every FD in §3.4.1 can be checked within a single relation, the decomposition is also dependency-preserving.

What is deliberately not a table. The prototype's records carried many derived counters: a problem's `status` and `solvedBy`, a sheet's `total`, `completed` and `assigned`, a user's `solved`, `streak` and `maxRating`, a contest's `status` and `participants`, and the entire leaderboard. Each is a function of the base relations, so storing it would introduce an update anomaly every time a submission arrived. They are implemented as the views in §3.5.

3.5. Appendix — SQL DDL and derived views

PostgreSQL dialect. The same script is kept in the repository as `docs/schema.sql`. Table names use snake_case; `app_user` is used because `user` is a reserved word.

```
-- O(1) ICPC preparation tracker — relational schema (PostgreSQL dialect).
-- Every relation below is in BCNF; see DB-Design.pdf §3.4 for the analysis.
-- Derived values (solve counts, streaks, sheet progress, leaderboard, contest
-- status, difficulty band) are NOT stored. They are computed by the views at
-- the end of this file so they can never drift from the base tables.

BEGIN;

-- ---------------------------------------------------------------- lookups
CREATE TABLE university (
  univ_id     SERIAL PRIMARY KEY,
  name        VARCHAR(120) NOT NULL UNIQUE,
  country     VARCHAR(60)  NOT NULL
);

CREATE TABLE topic (
  topic_id    SERIAL PRIMARY KEY,
  name        VARCHAR(60) NOT NULL UNIQUE
);

-- ---------------------------------------------------------------- people
CREATE TABLE app_user (
  user_id        SERIAL PRIMARY KEY,
  name           VARCHAR(80)  NOT NULL,
  email          VARCHAR(254) NOT NULL UNIQUE,
  password_hash  CHAR(60)     NOT NULL,
  role           VARCHAR(10)  NOT NULL CHECK (role IN ('student', 'mentor')),
  year_of_study  SMALLINT     CHECK (year_of_study BETWEEN 1 AND 5),
  univ_id        INT          NOT NULL REFERENCES university(univ_id),
  joined_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
  bio            VARCHAR(280),
  avatar_url     VARCHAR(500)
);

-- Weak entity: identified by (owner, platform). A handle is unique per platform.
CREATE TABLE judge_account (
  user_id      INT          NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
  platform     VARCHAR(20)  NOT NULL CHECK (platform IN ('codeforces', 'atcoder')),
  handle       VARCHAR(40)  NOT NULL,
  rating       INT,
  verified_at  TIMESTAMPTZ,
  PRIMARY KEY (user_id, platform),
  UNIQUE (platform, handle)
);

-- ---------------------------------------------------------------- problems
CREATE TABLE problem (
  problem_id     SERIAL PRIMARY KEY,
  platform       VARCHAR(20)  NOT NULL CHECK (platform IN ('codeforces', 'atcoder', 'manual')),
  external_code  VARCHAR(40)  NOT NULL,          -- e.g. '1234C'; for manual problems a slug
  title          VARCHAR(200) NOT NULL,
  rating         INT          CHECK (rating BETWEEN 800 AND 3500),
  url            VARCHAR(500) NOT NULL,
```

```
  has_editorial  BOOLEAN      NOT NULL DEFAULT FALSE,
  UNIQUE (platform, external_code)
);

CREATE TABLE problem_topic (
  problem_id  INT NOT NULL REFERENCES problem(problem_id) ON DELETE CASCADE,
  topic_id    INT NOT NULL REFERENCES topic(topic_id),
  PRIMARY KEY (problem_id, topic_id)
);

-- ------------------------------------------------------------- contests
CREATE TABLE contest (
  contest_id    SERIAL PRIMARY KEY,
  platform      VARCHAR(20)  NOT NULL CHECK (platform IN ('codeforces', 'atcoder')),
  external_id   VARCHAR(20)  NOT NULL,
  name          VARCHAR(200) NOT NULL,
  kind          VARCHAR(10)  NOT NULL CHECK (kind IN ('rated', 'gym', 'practice')),
  starts_at     TIMESTAMPTZ  NOT NULL,
  duration_min  INT          NOT NULL CHECK (duration_min > 0),
  UNIQUE (platform, external_id)
);

CREATE TABLE contest_problem (
  contest_id  INT     NOT NULL REFERENCES contest(contest_id) ON DELETE CASCADE,
  problem_id  INT     NOT NULL REFERENCES problem(problem_id),
  label       CHAR(2) NOT NULL,                  -- 'A', 'B', ... 'A1'
  PRIMARY KEY (contest_id, problem_id),
  UNIQUE (contest_id, label)
);

-- One row per judge submission, pulled by the sync job.
CREATE TABLE submission (
  submission_id           BIGSERIAL PRIMARY KEY,
  user_id                 INT          NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
  problem_id              INT          NOT NULL REFERENCES problem(problem_id),
  contest_id              INT          REFERENCES contest(contest_id),   -- NULL for practice
  verdict                 VARCHAR(20)  NOT NULL,   -- 'OK', 'WRONG_ANSWER',
'TIME_LIMIT_EXCEEDED', ...
  language                VARCHAR(40)  NOT NULL,
  submitted_at            TIMESTAMPTZ  NOT NULL,
  platform                VARCHAR(20)  NOT NULL,
  external_submission_id  BIGINT       NOT NULL,
  UNIQUE (platform, external_submission_id)
);
CREATE INDEX submission_user_time ON submission (user_id, submitted_at DESC);
CREATE INDEX submission_problem   ON submission (problem_id);

-- Rated participation; rating history is derived from this table.
CREATE TABLE participation (
  user_id     INT NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
  contest_id  INT NOT NULL REFERENCES contest(contest_id),
  old_rating  INT NOT NULL,
  new_rating  INT NOT NULL,
  rank        INT NOT NULL CHECK (rank > 0),
  PRIMARY KEY (user_id, contest_id)
);

-- ------------------------------------------------------------- sheets
CREATE TABLE sheet (
```

```
  sheet_id            SERIAL PRIMARY KEY,
  title               VARCHAR(120) NOT NULL,
  description         VARCHAR(500),
  author_id           INT          NOT NULL REFERENCES app_user(user_id),
  level               VARCHAR(6)   NOT NULL CHECK (level IN ('Easy', 'Medium', 'Hard')),
  deadline            DATE,
  created_at          TIMESTAMPTZ  NOT NULL DEFAULT now(),
  archive_contest_id  INT          REFERENCES contest(contest_id)   -- set for past-ICPC sheets
);

CREATE TABLE sheet_problem (
  sheet_id    INT      NOT NULL REFERENCES sheet(sheet_id) ON DELETE CASCADE,
  problem_id  INT      NOT NULL REFERENCES problem(problem_id),
  position    SMALLINT NOT NULL CHECK (position > 0),
  PRIMARY KEY (sheet_id, problem_id),
  UNIQUE (sheet_id, position)
);

CREATE TABLE sheet_assignment (
  sheet_id     INT         NOT NULL REFERENCES sheet(sheet_id) ON DELETE CASCADE,
  user_id      INT         NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
  assigned_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (sheet_id, user_id)
);

-- Manual tick-off for problems the sync cannot see (non-judge links).
-- Judge-hosted problems are marked complete by the v_sheet_progress view instead.
CREATE TABLE sheet_completion (
  sheet_id      INT         NOT NULL,
  user_id       INT         NOT NULL,
  problem_id    INT         NOT NULL,
  completed_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (sheet_id, user_id, problem_id),
  FOREIGN KEY (sheet_id, problem_id) REFERENCES sheet_problem(sheet_id, problem_id) ON DELETE
CASCADE,
  FOREIGN KEY (sheet_id, user_id)    REFERENCES sheet_assignment(sheet_id, user_id) ON DELETE
CASCADE
);

-- ------------------------------------------------------------------ teams
CREATE TABLE team (
  team_id      SERIAL PRIMARY KEY,
  name         VARCHAR(80)  NOT NULL,
  univ_id      INT          NOT NULL REFERENCES university(univ_id),
  coach_id     INT          NOT NULL REFERENCES app_user(user_id),
  invite_code  CHAR(10)     NOT NULL UNIQUE,
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
  UNIQUE (univ_id, name)
);

-- A user belongs to at most one team, hence user_id is the key.
CREATE TABLE team_member (
  user_id      INT         PRIMARY KEY REFERENCES app_user(user_id) ON DELETE CASCADE,
  team_id      INT         NOT NULL REFERENCES team(team_id) ON DELETE CASCADE,
  member_role  VARCHAR(40) NOT NULL,        -- 'Captain', 'DP & Math', ...
  joined_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ICPC rule: at most three members per team.
```

```
CREATE OR REPLACE FUNCTION enforce_team_size() RETURNS trigger AS $$
BEGIN
  IF (SELECT count(*) FROM team_member WHERE team_id = NEW.team_id) >= 3 THEN
    RAISE EXCEPTION 'team % already has three members', NEW.team_id;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER team_size_check
  BEFORE INSERT ON team_member
  FOR EACH ROW EXECUTE FUNCTION enforce_team_size();

-- ------------------------------------------------------------- resources
CREATE TABLE resource (
  resource_id   SERIAL PRIMARY KEY,
  title         VARCHAR(200) NOT NULL,
  type          VARCHAR(10)  NOT NULL CHECK (type IN ('Article', 'Video', 'Notes', 'Roadmap',
'Tutorial', 'Book')),
  url           VARCHAR(500) NOT NULL UNIQUE,
  topic_id      INT          NOT NULL REFERENCES topic(topic_id),
  level         VARCHAR(6)   NOT NULL CHECK (level IN ('Easy', 'Medium', 'Hard')),
  duration_min  INT          CHECK (duration_min > 0),
  added_by      INT          NOT NULL REFERENCES app_user(user_id)
);

CREATE TABLE bookmark (
  user_id        INT         NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
  resource_id    INT         NOT NULL REFERENCES resource(resource_id) ON DELETE CASCADE,
  bookmarked_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, resource_id)
);

-- ------------------------------------------------------------- calendar
-- Contest and sheet-deadline entries are derived; only these kinds are stored.
CREATE TABLE calendar_event (
  event_id      SERIAL PRIMARY KEY,
  title         VARCHAR(120) NOT NULL,
  kind          VARCHAR(12)  NOT NULL CHECK (kind IN ('practice', 'session', 'announcement')),
  starts_at     TIMESTAMPTZ  NOT NULL,
  duration_min  INT          NOT NULL CHECK (duration_min > 0),
  team_id       INT          REFERENCES team(team_id) ON DELETE CASCADE,   -- NULL = personal
  created_by    INT          NOT NULL REFERENCES app_user(user_id)
);

CREATE TABLE reminder (
  user_id         INT      NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
  event_id        INT      NOT NULL REFERENCES calendar_event(event_id) ON DELETE CASCADE,
  minutes_before  SMALLINT NOT NULL CHECK (minutes_before > 0),
  PRIMARY KEY (user_id, event_id)
);

-- ============================================================== derived data
-- Problem status per user: solved / attempted / todo.
CREATE VIEW v_problem_status AS
SELECT u.user_id, p.problem_id,
       CASE WHEN bool_or(s.verdict = 'OK') THEN 'solved'
            WHEN count(s.submission_id) > 0 THEN 'attempted'
            ELSE 'todo' END AS status
FROM app_user u CROSS JOIN problem p
```

```
LEFT JOIN submission s ON s.user_id = u.user_id AND s.problem_id = p.problem_id
GROUP BY u.user_id, p.problem_id;

-- Difficulty band is a function of rating, so it is derived rather than stored.
CREATE VIEW v_problem_difficulty AS
SELECT problem_id,
       CASE WHEN rating < 1400 THEN 'Easy'
            WHEN rating < 2000 THEN 'Medium'
            ELSE 'Hard' END AS difficulty
FROM problem;

-- Per-student sheet progress: synced solves OR manual ticks.
CREATE VIEW v_sheet_progress AS
SELECT sa.sheet_id, sa.user_id,
       count(sp.problem_id)                                          AS total,
       count(sp.problem_id) FILTER (WHERE ps.status = 'solved'
                                       OR sc.problem_id IS NOT NULL) AS completed
FROM sheet_assignment sa
JOIN sheet_problem sp       ON sp.sheet_id = sa.sheet_id
LEFT JOIN v_problem_status ps ON ps.user_id = sa.user_id AND ps.problem_id = sp.problem_id
LEFT JOIN sheet_completion sc ON sc.sheet_id = sa.sheet_id AND sc.user_id = sa.user_id
                             AND sc.problem_id = sp.problem_id
GROUP BY sa.sheet_id, sa.user_id;

-- Upsolve queue: contest problems the user entered but never solved.
CREATE VIEW v_upsolve_queue AS
SELECT pa.user_id, cp.contest_id, cp.problem_id, cp.label
FROM participation pa
JOIN contest_problem cp ON cp.contest_id = pa.contest_id
LEFT JOIN submission s  ON s.user_id = pa.user_id AND s.problem_id = cp.problem_id AND s.verdict =
'OK'
WHERE s.submission_id IS NULL;

-- Contest status is a function of the clock.
CREATE VIEW v_contest_status AS
SELECT contest_id,
       CASE WHEN now() < starts_at THEN 'upcoming'
            WHEN now() < starts_at + duration_min * INTERVAL '1 minute' THEN 'live'
            ELSE 'ended' END AS status
FROM contest;

COMMIT;
```